

LABORATORY REPORT: EXERCISE 3

Subject-Specific Socratic Learning with Reasoning LLMs

1. Large Language Models (LLMs) Used

To evaluate their effectiveness as personalized academic tutors, the following reasoning-enabled platforms were utilized:

DeepSeek-R1 (Deepthink enabled)

Qwen (Thinking enabled)

Grok (DeepSearch enabled)

2. Subject-by-Subject Playbook

A. Probability and Statistics (Focus: Intuition, Reasoning, and Visualisation)

Socratic Derivation: Instead of requesting direct answers, students can use a Socratic prompt to make the AI walk through a formula's derivation step-by-step, pausing to quiz the student at each stage to ensure conceptual comprehension.

Visualisations via Code: To cement statistical intuition (such as the Law of Large Numbers or t-distributions), students can prompt the AI to generate Python simulation code (using matplotlib and numpy) and execute it to watch the mathematical concepts in action.

Flawed-Logic Grader: Students can paste a solved problem and their steps, instructing the AI to act as a strict grader that only points out logical flaws line-by-line without giving away the final solution.

Practice Problem Generation: Students can prompt the AI to generate structured practice problems (easy, medium, and hard) with solutions hidden at the bottom to test their understanding actively.

B. Database Management Systems - DBMS (Focus: Relational Theory and SQL Mastery)

Normalization "Red Flag" Game: Students can paste an unnormalized database schema and prompt the AI to act as a senior Database Administrator (DBA). Instead of outputting the normalized tables, the AI asks guiding questions that challenge the student to identify redundancy and functional dependencies.

SQL Query Tuning: Rather than copy-pasting solutions, students can submit their draft SQL queries and ask the AI to explain the execution order (e.g., FROM, WHERE, GROUP BY) and suggest optimizations.

Interactive SQL Practice: Students can prompt the AI to simulate an interviewer, generating relational schemas (like e-commerce tables) and testing the student's ability to write window functions or joins.

Database Disaster Recovery Drills: To master theoretical ACID properties, students can prompt the AI to simulate transactional system crashes at different milliseconds and explain how the recovery manager ensures durability.

C. Software Engineering (Focus: System Design, Architecture, and Lifecycle)

Agile Sprint Simulator: To practice requirements engineering, students can prompt the AI to act as a Product Owner, generating structured User Stories and organizing them into prioritized sprints.

UML Reverse Engineering: Students can submit legacy code blocks and ask the AI to generate high-level architectural visualizations using Mermaid.js class or sequence diagrams to evaluate system coupling and cohesion.

SOLID Code Reviewer: Students can submit their code modules and ask the AI to review them for adherence to SOLID principles and security vulnerabilities, ranking recommendations from "Critical Bug" to "Nitpick".

Methodology Trade-Off Analysis: Students can prompt the AI to create scenarios comparing Agile and Waterfall models to understand when one methodology is strictly superior over the other.

D. Universal Meta-Prompts

The Feynman Technique: Students can explain a topic (such as indexing) to the AI and instruct it to act as a 10-year-old, interrupting whenever jargon is used and demanding simpler analogies.

The Staircase Difficulty Prompt: The AI is prompted to generate three questions of ascending difficulty (definition, application, and a conceptual trick question) and withhold the solutions until the student attempts them.

The Exam Generator: The AI acts as a strict professor, generating a short-answer exam on a chosen topic and grading the student's submissions harshly to expose knowledge gaps.

3. Reflection: Best Practices & Pitfalls

Overcoming the "Illusion of Competence": Merely reading a well-structured explanation generated by an AI creates a false sense of mastery. Students do not truly understand a concept until they can close the chat and solve the problem independently.

Mathematical and Syntax Verification: Because LLMs are language models rather than native calculators, they are prone to mathematical hallucinations. All steps and query structures must be verified using course textbooks, local SQL compilers, or Python simulations.

No Paste-and-Submit: To preserve academic integrity, students must treat the AI as a Socratic conversational partner and helper, never using it to write entire assignments or projects.

4. Comparative Evaluation: Which Tool Produced the Best Results?

DeepSeek-R1 (Deepthink): This tool proved to be the most powerful for Probability and Statistics. Its transparent chain-of-thought allows students to inspect the logical steps and proofs behind complex mathematical derivations before reaching the final output.

Qwen (Thinking Mode): Demonstrated exceptional structural capability for DBMS and Software Engineering tasks. It consistently generated clean, error-free Mermaid.js architectural diagrams and highly organized SQL query templates.

Grok (DeepSearch): Excelled at gathering real-world case studies and analyzing modern industry practices for Software Engineering. Its deep web search capabilities ensured that development lifecycles and tool comparisons remained up-to-date.

Conclusion: For core computer science and mathematical subjects, DeepSeek-R1 offered the most rigorous, Socratic, and mathematically precise dialogue, making it the most reliable tool for academic learning in this lab guide.



How can a university student use generative AI to learn Probability and Statistics, Database Management Systems (DBMS), and Software Engineering effectively?

Use generative AI as an interactive tutor, explainer, problem generator, code assistant, and feedback tool—while always verifying outputs, practicing actively, and respecting academic integrity.

AI tools (Grok, ChatGPT, Claude, Gemini, etc.) excel at personalized explanations, generating varied examples, breaking down complex topics, simulating discussions, and iterating on your work. Treat them as a tireless study partner rather than a replacement for textbooks, lectures, coding practice, or your own reasoning.

Core Best Practices for All Three Subjects

- **Be specific and iterative:** Provide context (e.g., "Explain conditional probability for a second-year CS student who knows basic set theory, using a real-world example involving spam filters"). Ask follow-ups, request alternative explanations, analogies, or visualizations described in text.
- **Demand step-by-step reasoning and verification:** Always ask the AI to show work. Then re-derive it yourself, cross-check with your textbook/notes, or test code/outputs.
- **Active learning loop:** Generate problems → attempt them first → compare with AI solutions → ask the AI to critique *your* solution → generate variations.
- **Generate study materials:** Summaries of chapters, flashcards (Anki-style), quizzes with answers hidden until requested, concept maps, or spaced-repetition schedules.
- **Project and application focus:** Ask for mini-projects that integrate the topics (e.g., a stats-backed database-backed software system).
- **Limitations awareness:** AI can hallucinate formulas, edge cases, or outdated practices. Cross-reference with primary sources (textbooks like Ross for probability, Silberschatz/Korth for DBMS,

Qwen3.7-Plus

New Chat

Search Chats

Community

Coder

Projects

New Project

All chats

Today

AI-Powered Learning Strategies ...

Thinking completed

Using generative AI as a learning tool can be transformative for a university student, acting as a 24/7 personalized tutor, study partner, and code reviewer. However, the key to effectiveness is using AI to **guide your thinking** rather than **do the work for you**.

Here is a comprehensive, subject-specific guide on how to leverage generative AI to master Probability and Statistics, Database Management Systems (DBMS), and Software Engineering, along with best practices to ensure academic integrity and deep learning.

General Best Practices for Learning with AI

1. Use the Socratic Method: Prompt the AI to *guide* you, not give you the answer.
Example: "I'm stuck on this statistics problem. Don't give me the answer. Ask me guiding questions to help me figure out the next step."

2. Verify Everything: AI can hallucinate, especially with complex math or niche database edge cases. Cross-reference with your textbook, lecture notes, or official documentation.

3. Iterative Prompting: Treat the AI like a conversation. If an explanation is too dense, say, "Explain that like I'm a beginner," or "Give me a real-world analogy."

4. Maintain Academic Integrity: Use AI to understand concepts, generate practice problems, or debug your own code. Never submit AI-generated text/code as your own final assignment unless

Ask Qwen

Thinking

DHANU DHANUKSHAN

30°C Partly su...

21:56

deepseek

New chat

Today

AI for Stats DBMS SE

AI for Stats DBMS SE
Instant

How can a university student use generative AI to learn Probability and Statistics, Database Management Systems (DBMS), and Software Engineering effectively?

Thought for 4 seconds

1. Deconstruct the User Request:

Target audience: University student.

Goal: Use generative AI effectively to learn three specific subjects: Probability & Statistics, Database Management Systems (DBMS), and Software Engineering.

Constraint: Must be practical, actionable, and tailored to each subject's unique nature.

2. Analyze the Subjects & Their Learning Modes:

Probability & Statistics: Math-heavy, conceptual, requires step-by-step derivations, visualization, and problem-solving practice. Prone to "hallucinated" numbers, so must emphasize verification.

DBMS: Conceptual (ACID, normalization, indexing) + practical (SQL, query optimization, NoSQL). Requires schema design and query writing.

Software Engineering: Process-heavy (SDLC, Agile, requirements), design (UML, architecture), and coding (best practices, testing, refactoring). Very project-oriented.

3. Brainstorm Generic AI Use Cases (filter for effectiveness):

Personalized tutor (Socratic method).

Practice problem generator.

Code/SQL debugger.

Summarizer of complex texts.

Project planner/mentor.

Interview/quiz preparer.

Message DeepSeek

DeepThinkSearch

AI-generated, for reference only

DHANU DHANUKSHAN

30°C Partly cloudy 21:37